JAVA FREQUENCY COUNTING

Brute Force • Frequency Array • HashMap • Sorting • 1D/2D Arrays • Strings • DSA Patterns

Complete learning guide: Ek hi technique nahi—frequency/counting problems ko solve karne ke major approaches, unki conditions, Java code, dry runs aur complexity.

1. Frequency counting ka basic meaning

Frequency ka matlab hai: input mein koi value kitni baar aayi. Example: [4, 2, 2, 7, 2, 4] mein 2 ki frequency 3, 4 ki frequency 2 aur 7 ki frequency 1 hai.

```
Input:      [4, 2, 2, 7, 2, 4]
Distinct:    2   4   7
Frequency:   3   2   1
```

Frequency counting duplicates, missing values, anagrams, first unique character, majority element, top-K frequent values aur pair-counting jaise questions mein use hoti hai.

2. Sabse pehle: kaunsa approach kab use karein?

Approach	Kab use karein	Typical time
Brute force	Small input; har element ko doosron se compare/count karna	$O(m^2)$
Frequency array	Values small, non-negative ya compact known range mein hon	O(m + R)
HashMap	Values negative, huge, sparse ya unknown range mein hon	Expected O(m)
Sorting	Input reorder kar sakte ho; equal values ko saath lana hai	O(m log m)

Notation: m = input mein total elements; R = frequency array mein represent ki gayi value range ka size; u = distinct values ki sankhya.

3. Approach 1 — Brute force frequency counting

Har element ke liye poore array mein count karo. Simple hai, lekin repeated comparisons ki wajah se slow ho sakta hai.

```
int[] arr = {4, 2, 2, 7, 2, 4};

for (int i = 0; i < arr.length; i++) {
    // Same value pehle count ho chuki ho to skip
    boolean alreadyCounted = false;

    for (int k = 0; k < i; k++) {
        if (arr[k] == arr[i]) {
            alreadyCounted = true;
            break;
        }
    }
    if (alreadyCounted) continue;

    int count = 0;
    for (int j = 0; j < arr.length; j++) {
        if (arr[i] == arr[j]) {
            count++;
        }
    }

    System.out.println(arr[i] + " -> " + count);
}
```

Output (order input ke hisaab se): 4 -> 2, 2 -> 3, 7 -> 1. Time $O(m^2)$, auxiliary space O(1) (output ko ignore karke).

Brute force kab? Jab input chhota ho, concept seekhna ho, ya extra data structure avoid karna ho. Large input ke liye usually better approach choose karo.

4. Approach 2 — Frequency array (direct indexing)

Jab values ki range chhoti aur known ho, value ko array index bana do. Har value ka count freq[value] mein store hota hai.

4A. Values 0 se maxValue tak

```
int[] arr = {0, 1, 1, 2, 3};
int maxValue = 3;

int[] freq = new int[maxValue + 1]; // indexes 0,1,2,3
for (int value : arr) {
    freq[value]++;
}
System.out.println(java.util.Arrays.toString(freq));
// [1, 2, 1, 1]
```

4B. Values 1 se maxValue tak

```
int[] arr = {1, 2, 2, 3, 4};
int maxValue = 4;

int[] freq = new int[maxValue + 1]; // index 0 unused
for (int value : arr) {
    freq[value]++;
}
System.out.println(java.util.Arrays.toString(freq));
// [0, 1, 2, 1, 1]
```

Index 4 access karna hai, isliye length 5 chahiye. Index 0 unused hai, lekin array ka valid slot hai. General rule: direct value-as-index mapping mein length = maxValue + 1.

4C. 1 se maxValue tak, lekin extra slot nahi chahiye

```
int[] freq = new int[maxValue]; // length 4; indexes 0..3
for (int value : arr) {
    freq[value - 1]++;
}
// value 1 -> index 0; value 4 -> index 3
```

4D. Negative values ke liye offset

```
int[] arr = {-2, -1, -1, 0, 2};
int minValue = -2, maxValue = 2;
int[] freq = new int[maxValue - minValue + 1];

for (int value : arr) {
    int index = value - minValue; // value + 2
    freq[index]++;
}
// indexes represent -2, -1, 0, 1, 2
// [1, 2, 1, 0, 1]
```

Formula: **index = value - minValue**; **size = maxValue - minValue + 1**. Range bahut wide ho to array allocate mat karo—HashMap better ho sakta hai.

5. Approach 3 — HashMap frequency

HashMap use karo jab values arbitrary, negative, huge ya sparse hon. Ye sirf input mein present distinct keys store karta hai.

```
import java.util.HashMap;
import java.util.Map;

int[] arr = {1000000, -7, 1000000, 42, -7};
Map<Integer, Integer> freq = new HashMap<>();

for (int value : arr) {
    freq.put(value, freq.getOrDefault(value, 0) + 1);
}
System.out.println(freq);
// {1000000=2, -7=2, 42=1} (display order guaranteed nahi)
```

getOrDefault(key, 0): key maujood hai to existing count, warna 0. Usmein 1 add karke update kar do.

HashMap ko traverse karna

```
for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
    int value = entry.getKey();
    int count = entry.getValue();
    System.out.println(value + " -> " + count);
}
```

Expected time O(m), extra space O(u). HashMap order guaranteed nahi hota. Agar sorted key order chahiye, TreeMap use kar sakte ho (operations O(log u)).

6. Approach 4 — Sorting se frequency

Sort karne par equal values paas-paas aa jaati hain. Phir consecutive group ka count nikaal sakte ho.

```
import java.util.Arrays;

int[] arr = {4, 2, 2, 7, 2, 4};
Arrays.sort(arr); // [2, 2, 2, 4, 4, 7]

int i = 0;
while (i < arr.length) {
    int j = i;
    while (j < arr.length && arr[j] == arr[i]) {
        j++;
    }

    int count = j - i;
    System.out.println(arr[i] + " -> " + count);
    i = j;
}
```

Time O(m log m) for sorting plus O(m) scan. Sorting original array ka order change karti hai. Extra space Java sorting implementation aur primitive/object array par depend kar sakti hai.

7. Frequency array ka size — full decision table

Allowed values	Array size	Index mapping
0 to n-1	n	index = value
1 to n	n+1	index = value
1 to n, compact alternative	n	index = value-1
0 to max	max+1	index = value
min to max (negative possible)	max-min+1	index = value-min
Huge / sparse / unknown	Prefer HashMap	key = value

Common mistake: freq array ka size arr.length se blindly decide mat karo. Input length batata hai kitne elements hain; value range batati hai direct-index frequency array kitna bada chahiye.

8. Java default values — array zero se initialize hota hai

```
int[] freq = new int[5];
System.out.println(java.util.Arrays.toString(freq));
// [0, 0, 0, 0, 0]
```

Java ke new int[size] ke saare slots default 0 hote hain. Isliye count badhane se pehle manually zero fill karne ki zarurat nahi.

Type	Default
int	0
long	0L
float	0.0f
double	0.0
boolean	false
char	'\\u0000'
Reference types (String etc.)	null

9. Complete LeetCode problem — Find Missing and Repeated Values

Grid n x n hai; possible values 1 se n^2. Ek value twice aati hai aur ek absent hai. Return [repeated, missing].

```
import java.util.Arrays;
```

```
public class Main {

    public static int[] findMissingAndRepeatedValues(int[][] grid) {
        int n = grid.length;
        int[] freq = new int[n * n + 1];

        // Count values in the 2D grid
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                freq[grid[i][j]]++;
            }
        }

        int repeated = -1;
        int missing = -1;

        for (int value = 1; value <= n * n; value++) {
            if (freq[value] == 2) {
                repeated = value;
            } else if (freq[value] == 0) {
                missing = value;
            }
        }

        return new int[]{repeated, missing};
    }

    public static void main(String[] args) {
        int[][] grid = {
            {1, 3},
            {2, 2}
        };

        int[] ans = findMissingAndRepeatedValues(grid);
        System.out.println(Arrays.toString(ans)); // [2, 4]
    }
}
```

Why n² + 1? Values 1..n² ko directly index banaya hai. Highest required index n² hai; us index ko hold karne ke liye array length n²+1 honi chahiye. Index 0 unused hai.

Time: O(n²) count + O(n²) scan = O(n²). **Auxiliary space:** O(n²).

10. 2D array mein frequency

2D matrix ke har cell ko nested loops se visit karo. Agar matrix rows x columns ka hai, total cells rows*columns hain. Square n x n matrix mein n² cells.

```
int[][] grid = {
    {1, 2, 2},
    {3, 1, 4}
};
int[] freq = new int[5]; // values 0..4

for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        freq[grid[r][c]]++;
    }
}
```

Agar 2D matrix ke values ki range unknown / wide ho, HashMap mein har grid[r][c] ko key bana kar count karo.

11. Strings / characters ki frequency

11A. Sirf lowercase English a-z

```
String s = "banana";
int[] freq = new int[26];

for (int i = 0; i < s.length(); i++) {
    char ch = s.charAt(i);
    freq[ch - 'a']++;
}
// a=3, b=1, n=2
```

Ye approach tabhi valid hai jab input guaranteed lowercase a-z ho.

11B. General characters with HashMap

```
import java.util.HashMap;
import java.util.Map;

String s = "Hello!";
Map<Character, Integer> freq = new HashMap<>();

for (char ch : s.toCharArray()) {
    freq.put(ch, freq.getOrDefault(ch, 0) + 1);
}
```

Java char UTF-16 code unit hai. Full Unicode code points count karne ke liye String.codePoints() aur Map use kar sakte ho.

12. Common DSA problems — kis approach se solve hote hain?

Problem	Common frequency approach
Find duplicates	Frequency array / HashMap; count > 1
Find missing value in known range	Frequency array; count == 0
Find single / unique values	Frequency count == 1; XOR is another option in special constraints
Anagram check	Character frequency array / HashMap; compare counts
First non-repeating character	First pass count, second pass original order; count == 1
Majority element	HashMap/frequency; Boyer-Moore is O(1) extra space
Mode (most frequent)	Frequency array / HashMap; maximum count
Top K frequent elements	HashMap counts + heap or bucket sort
Pair sum / Two Sum variants	HashMap of seen values or counts
Subarray sum equals K	Prefix sum + HashMap frequency
Count pairs with a target sum	Frequency map; carefully handle equal-value pairs

13. Anagram check — frequency array example

```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;

    int[] freq = new int[26];

    for (int i = 0; i < a.length(); i++) {
        freq[a.charAt(i) - 'a']++;
        freq[b.charAt(i) - 'a']--;
    }

    for (int count : freq) {
        if (count != 0) return false;
    }
    return true;
}
// Assumes lowercase English letters a-z.
```

Idea: pehle string ke characters add (+1), doosri ke characters subtract (-1). Agar saare counts zero, dono strings anagram hain.

14. Complexity quick reference

Approach	Time	Extra space
Brute force count each distinct value	$O(m^2)$	O(1)
Frequency array	O(m + R)	O(R)
HashMap	Expected O(m)	O(u)
Sorting + grouped scan	O(m log m)	Sort implementation dependent
2D matrix traversal	O(rows * cols)	Depends on frequency structure

Approach	Time	Extra space
Lowercase a-z count	O(m + 26) = O(m)	O(26) = O(1)

15. Interview checklist — code likhne se pehle ye sawal poochho

• Values ki minimum aur maximum range kya hai?

• Kya values non-negative aur compact range mein hain? Frequency array use ho sakta hai?

• Kya negative / huge / sparse values hain? HashMap consider karo.

• Kya input reorder kar sakte ho? Sorting approach possible hai?

• Kya same value ko baar-baar count karne se bachna hai? Brute force mein alreadyCounted check karo.

• Kya output order important hai? HashMap order guaranteed nahi; LinkedHashMap / TreeMap ki need ho sakti hai.

• Kya array ke values index ban rahe hain? Highest index ke liye size = highestIndex + 1 hona chahiye.

• Kya Java int[] ke zero initialization ka benefit le rahe ho? Manually zero fill mat karo.

Final rule to remember: Brute force = compare/count manually; Frequency array = compact known range; HashMap = arbitrary values; Sorting = equal values ko side-by-side laana. Problem ki constraints dekh kar approach choose karo.